codex-windows / 019f2174-3a7a-7c73-8cb3-474b86987ac6

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-02T11-41-33-019f2174-3a7a-7c73-8cb3-474b86987ac6.jsonl`
- SHA-256: `110efe4576a3191fe5de5673ed609214ccb36f1e7e2a06dc6130e554d65725a9`
- Source modified: `2026-07-02T06:11:33+00:00`
- Imported at: `2026-07-05T16:48:16+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\badminton-highlight-indexer`
- model_provider: `openai`
- session_id: `019f2174-3a7a-7c73-8cb3-474b86987ac6`
- source: `vscode`

Transcript

1. user (2026-07-02T06:11:33.115Z)

In backend/compute/cloud_run.py, GoogleCloudRunJobClient.run_job calls `client.job_path(project, region, job_name)` where `project` is `Optional[str]` (from `GOOGLE_CLOUD_PROJECT` env, can be None) but `JobsClient.job_path` expects `str`. mypy flags this (only when google-cloud-run is installed, so CI ? which doesn't install it ? stays green, but it's a real latent bug: a None project would fail the real Cloud Run dispatch). Fix by validating project is set (raise a clear error if GOOGLE_CLOUD_PROJECT is unset) before calling job_path, or narrowing the type. Add a small test for the missing-project case. Keep it minimal; run ruff + `mypy backend` (with google-cloud-run installed) + the compute tests.

2. assistant (2026-07-02T06:11:33.115Z)

I'll start by understanding the current state and reading the target file. Let me gather context in parallel.

3. assistant (2026-07-02T06:11:33.115Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py
[/external_agent_tool_call]

4. assistant (2026-07-02T06:11:33.115Z)

[external_agent_tool_result]
```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4 GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero (D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
```

```
    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` + native detector)
    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just computes.

    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole shim is unit-tested
    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run`` and is
    owner-verified on the M7 real run.
    """

    from __future__ import annotations

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, List, Optional

    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy, poll_until
    from backend.storage import fetch, parse_uri
    from backend.storage import get_storage

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-polls)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test dependency)."""

        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
```

```
66              try:
67                  from google.cloud import run_v2  # type: ignore[attr-defined]
68              except Exception as exc:  # pragma: no cover - real SDK not present in CI
69                  raise RuntimeError(
70                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                      "control plane (it is intentionally NOT a CI/test dependency)."
72                  ) from exc
73              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
74              name = client.job_path(project, region, job_name)
75              overrides = run_v2.RunJobRequest.Overrides(
76                  contai...
[/external_agent_tool_result]
```

5. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_call: Bash]
description: Check git status and current branch
command: git status --short && echo "---BRANCH---" && git branch --show-current && echo "---
HEAD---" && git log --oneline -1
[/external_agent_tool_call]
```

6. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_call: Glob]
input: {"pattern":"backend/compute/**"}
[/external_agent_tool_call]
```

7. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_call: Glob]
input: {"pattern":"**/test*compute*"}
[/external_agent_tool_call]
```

8. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_result]
backend\compute\__init__.py
backend\compute\cloud_run.py
backend\compute\base.py
backend\compute\__pycache__\__init__.cpython-313.pyc
backend\compute\__pycache__\base.cpython-313.pyc
backend\compute\__pycache__\cloud_run.cpython-313.pyc
[/external_agent_tool_result]
```

9. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_result]
tests\test_compute_backend.py
tests\__pycache__\test_compute_backend.cpython-313-pytest-9.0.3.pyc
[/external_agent_tool_result]
```

10. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_result]
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---BRANCH---
---HEAD---
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
[/external_agent_tool_result]
```

11. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py
[/external_agent_tool_call]
```

12. assistant (2026-07-02T06:11:33.115Z)

```
[external_agent_tool_result]
1       """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3       All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4       worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5       ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6       selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7       """
8       import json
9       import os
10      import tempfile
11
12      from backend.compute import ComputeJobSpec, get_compute_backend
13      from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
14      from backend.config import AppConfig
15      from backend.infrastructure.execution_policy import ExecutionPolicy
16
17      # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
18      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
19
20
21      class _FakeRunClient(CloudRunJobClient):
22          """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
23          exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
24
25          def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
26              self.calls: list[dict] = []
27              self.video_id, self.segment_count, self.returncode = video_id, segment_count,
returncode
28
29          def run_job(self, job_name, argv, *, region, project=None):
30              self.calls.append({"job": job_name, "argv": list(argv), "region": region,
"project": project})
31              out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
32              done_uri = argv[argv.index("--done-uri") + 1]
33              outputs = os.path.join(out_uri, "outputs", self.video_id)
34              os.makedirs(outputs, exist_ok=True)
35              with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
36                  json.dump({"video_id": self.video_id, "segment_count": self.segment_count,
37                             "status": "success"}, f)
38              os.makedirs(os.path.dirname(done_uri), exist_ok=True)
39              with open(done_uri, "w", encoding="utf-8") as f:
40                  json.dump({"video_id": self.video_id, "returncode": self.returncode,
41                             "segment_count": self.segment_count, "status": "success"}, f)
42              return "exec-123"
43
44
45      class _NoopRunClient(CloudRunJobClient):
46          """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
```

```
47          directly (a gs:// out_uri has no local dir to makedirs)."""
48
49          def __init__(self):
50              self.calls: list[dict] = []
51
52          def run_job(self, job_name, argv, *, region, project=None):
53              self.calls.append({"job": job_name, "argv": list(argv), "region": region,
"project": project})
54              return "exec"
55
56
57      # ---------------------------------------------------------------- factory (default-
OFF)
58
59      def _cfg(compute_target="", storage_backend="local"):
60          return AppConfig.model_validate({
61              "deployment": {"compute_target": compute_target},
62              "storage": {"backend": storage_backend},
63          })
64
65
66      def test_factory_is_none_for_local_targets():
67          """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
68          assert get_compute_backend(_cfg("")) is None
69          assert get_compute_backend(_cfg("local_gpu")) is None
70          assert get_compute_backend(_cfg("cpu_mock")) is None
71
72
73      def test_factory_returns_cloud_run_for_cloud_target():
74          backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
75          assert isinstance(backend, CloudRunCompute)
76
77
78      # ---------------------------------------------------------- CloudRunCompute dispatch
+ poll
79
80      def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
81          cli...
[/external_agent_tool_result]
```

13. assistant (2026-07-02T06:11:33.115Z)

Let me look at the factory to understand how `project` flows from `GOOGLE_CLOUD_PROJECT`.

14. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\__init__.py
[/external_agent_tool_call]
```

15. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_result]
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
```

```
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38      ) -> Optional[ComputeBackend]:
39          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40          in-process (the default-OFF path).
41
42          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46          """
47          target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48          if target != "cloud_run":
49              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
[/external_agent_tool_result]
```

16. assistant (2026-07-02T06:11:33.116Z)

I now fully understand the flow:

- `get_compute_backend` ([backend/compute/__init__.py:56](backend/compute/__init__.py)) reads `project = os.environ.get("GOOGLE_CLOUD_PROJECT")` ? `None` if unset.
- That flows to `CloudRunCompute._project` ? `run_job(project=...)` ? `client.job_path(project, region, job_name)` ([backend/compute/cloud_run.py:74](backend/compute/cloud_run.py)), where `job_path` requires `str`.

The fix: guard `project` at the top of `run_job` (before the lazy SDK import, so the guard is testable in CI where `google-cloud-run` is absent). This both narrows `Optional[str]` ? `str` for mypy and makes a misconfigured dispatch fail loudly.

Let me check the lint/type config and whether the SDK is installed, then make the edit.

17. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_call: Bash]
description: Inspect ruff/mypy configuration
command: echo "=== ruff/mypy config (pyproject) ===" && (cat pyproject.toml 2>/dev/null | sed -n '1,200p') ; echo "=== setup.cfg ===" && (cat setup.cfg 2>/dev/null) ; echo "=== mypy.ini ===" && (cat mypy.ini 2>/dev/null) ; echo "=== ruff.toml ===" && (cat ruff.toml 2>/dev/null)
[/external_agent_tool_call]

18. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_result]
=== ruff/mypy config (pyproject) ===

PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).

#

Build backend: hatchling. `backend/` is an intentional NAMESPACE package (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's auto-detection would otherwise miss the un-__init__'d `backend` root.

#

torch/torchvision are deliberately NOT listed in [project.dependencies]: their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url), which PEP 621 cannot express. Install them separately ? see [project.optional-depend `gpu` (documentation only) and the README. CI installs CPU torch on its own line.

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "badminton-highlight-indexer"
version = "0.1.0"
description = "AI-assisted rally segmentation and highlight indexing for racket sports."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
```

Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).

```
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
```

```
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
    "google-genai>=2.4.0",
    "supervision>=0.21.0,<0.30.0",
    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d /
    # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a supervision
    # bump or a pruned-transitive freeze could silently break inference. Declare it. (BSD-3.)
    "scipy>=1.10",
]

[project.optional-dependencies]
```

Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).

```
dev = [
    "pytest",
    "pytest-cov",
    "httpx",        # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]
```

M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_a `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/mul

```
auth = ["PyJWT[crypto]>=2.8"]
```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Pinned to its git tag per BACKLOG. The private repo is fetched over SSH; if you lack access, simply omit this group ? reporting tests skip via pytest.importorskip. For local dev against a sibling checkout: pip install -e ../sports-obsreport --no-build-isolation

```
reporting = [
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]
```

`gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot express, so they are NOT declarable here. Install them out-of-band instead: # CUDA 12.4 box: pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org # CPU-only box (also what CI uses): pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

```
gpu = []
```

Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs these. Cloud SDKs are imported lazily, so `import backend.storage` works without the install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storag

```
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

[project.scripts]
```

`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()

```
indexer = "backend.main:main...
[/external_agent_tool_result]
```

19. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_call: Bash]
description: Check for google-cloud-run, mypy, ruff availability
command: python -c "import google.cloud.run_v2; print('run_v2 OK',
google.cloud.run_v2.__file__)" 2>&1 | head -5 ; echo "---" ; python -c "import mypy;
print('mypy', mypy.__version__)" 2>&1 | head -2 ; echo "---" ; python -c "import ruff" 2>&1 |
head -2 ; ruff --version 2>&1 | head -1
[/external_agent_tool_call]
```

20. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_result]
run_v2 OK C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2\__init__.py
---
Traceback (most recent call last):
  File "<string>", line 1, in <module>
---
ruff 0.15.16
[/external_agent_tool_result]
```

21. assistant (2026-07-02T06:11:33.116Z)

`google-cloud-run` (run_v2) is already installed and `cloud_run.py` is NOT in the mypy quarantine list ? so mypy checks it. Let me confirm the mypy invocation CI uses, verify mypy runs, and **reproduce the error before fixing** (baseline).

22. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_call: Bash]
description: Check mypy version and CI lint/type invocations
command: python -m mypy --version 2>&1 | head -2; echo "=== CI mypy/ruff invocation ==="; grep
-nE "mypy|ruff" .github/workflows/*.yml 2>/dev/null | head -40
[/external_agent_tool_call]
```

23. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_result]
mypy 2.1.0 (compiled: yes)
=== CI mypy/ruff invocation ===
16:  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17:  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
28:      - name: Install ruff
29:        run: python -m pip install ruff
31:        run: ruff check . --output-format github
48:  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
50:  # up. Typed core deps are installed so mypy sees their real signatures
60:      - name: Install mypy + typed core deps
61:        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
63:        run: mypy backend training
102:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
[/external_agent_tool_result]
```

24. assistant (2026-07-02T06:11:33.116Z)

mypy 2.1.0 is available, CI runs `mypy backend training` (and notably does **not** install `google-cloud-run`, confirming why CI stays green). Let me reproduce the latent error locally where the SDK *is* installed:

25. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_call: Bash]
description: Reproduce the mypy error on cloud_run.py
command: python -m mypy backend/compute/cloud_run.py 2>&1 | head -30
[/external_agent_tool_call]

26. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_result]
Success: no issues found in 1 source file
[/external_agent_tool_result]

27. assistant (2026-07-02T06:11:33.116Z)

Single-file check resolves imports differently. Let me run the exact CI invocation `mypy backend` and filter for the relevant line:

28. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_call: Bash]
description: Run full mypy backend and filter for cloud_run errors
command: python -m mypy backend 2>&1 | grep -iE "cloud_run|job_path|Error|error:" | head -30; echo "=== full tail ==="; python -m mypy backend 2>&1 | tail -5
[/external_agent_tool_call]

29. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_result]
=== full tail ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 133 source files
[/external_agent_tool_result]

30. assistant (2026-07-02T06:11:33.116Z)

mypy reports success even with the SDK installed. Let me investigate the actual `job_path` signature and whether the package ships type info (`py.typed`):

31. assistant (2026-07-02T06:11:33.116Z)

[external_agent_tool_call: Bash]
description: Inspect job_path signature and py.typed marker
command: python - <<'PY'
import inspect, os
from google.cloud import run_v2
from google.cloud.run_v2 import JobsClient
print("run_v2 version:", getattr(run_v2, "__version__", "n/a"))
print("job_path sig:", inspect.signature(JobsClient.job_path))

find py.typed

```
pkgdir = os.path.dirname(inspect.getfile(run_v2))
root = os.path.dirname(os.path.dirname(os.path.dirname(pkgdir)))  # google/cloud/run_v2 -> up
for base in [pkgdir, os.path.dirname(pkgdir)]:
    pt = os.path.join(base, "py.typed")
    print("py.typed at", base, "=>", os.path.exists(pt))
```

Show the source of job_path

```
src = inspect.getsource(JobsClient.job_path)
print("----- job_path source -----")
print(src[:600])
PY
[/external_agent_tool_call]
```

32. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_result]
run_v2 version: 0.16.0
job_path sig: (project: str, location: str, job: str) -> str
py.typed at C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2 => True
py.typed at C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud => False
----- job_path source -----
    @staticmethod
    def job_path(
        project: str,
        location: str,
        job: str,
    ) -> str:
        """Returns a fully-qualified job string."""
        return "projects/{project}/locations/{location}/jobs/{job}".format(
            project=project,
            location=location,
            job=job,
        )
[/external_agent_tool_result]
```

33. assistant (2026-07-02T06:11:33.116Z)

`job_path(project: str, ...)` is strictly typed and `run_v2` ships `py.typed`. Yet `mypy backend` passes. The reason is the `# type: ignore[attr-defined]` on the import line ? let me confirm whether that's masking the error, since I want to be sure the fix actually resolves what mypy sees. Let me probe with `reveal_type`:

34. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_call: Bash]
description: Probe mypy view of run_v2 with and without type-ignore
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && mkdir -p scratch && cat >
scratch/_mypy_probe.py <<'PY'
from typing import Optional, List

def probe(region: str, job_name: str, project: Optional[str] = None) -> str:
    from google.cloud import run_v2  # type: ignore[attr-defined]
    reveal_type(run_v2)  # noqa
    client = run_v2.JobsClient()
    reveal_type(client.job_path)  # noqa
    name = client.job_path(project, region, job_name)
    return name

def probe_no_ignore(region: str, job_name: str, project: Optional[str] = None) -> str:
    from google.cloud import run_v2
    reveal_type(run_v2)  # noqa
    client = run_v2.JobsClient()
    reveal_type(client.job_path)  # noqa
```

```
    name = client.job_path(project, region, job_name)
    return name
PY
python -m mypy scratch/_mypy_probe.py 2>&1 | head -40
[/external_agent_tool_call]
```

35. assistant (2026-07-02T06:11:33.116Z)

```
[external_agent_tool_result]
scratch\_mypy_probe.py:5: note: Revealed type is "types.ModuleType"
scratch\_mypy_probe.py:7: note: Revealed type is "def (project: str, location: str, job: str) ->
str"
scratch\_mypy_probe.py:8: error: Argument 1 to "job_path" of "JobsClient" has incompatible type
"str | None"; expected "str"  [arg-type]
scratch\_mypy_probe.py:13: note: Revealed type is "types.ModuleType"
scratch\_mypy_probe.py:15: note: Revealed type is "def (project: str, location: str, job: str)
-> str"
scratch\_mypy_probe.py:16: error: Argument 1 to "job_path" of "JobsClient" has incompatible type
"str | None"; expected "str"  [arg-type]
Found 2 errors in 1 file (checked 1 source file)
[/external_agent_tool_result]
```

36. assistant (2026-07-02T06:11:33.117Z)

mypy **does** flag it: `Argument 1 to "job_path"... incompatible type "str | None"; expected "str"`. But the broad `mypy backend` run passed ? I need to reconcile that before I trust my verification. Let me re-run `mypy backend` freshly (no cache) and target the module directly:

37. assistant (2026-07-02T06:11:33.117Z)

```
[external_agent_tool_call: Bash]
description: Fresh mypy backend run and module-targeted check
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== mypy backend
--no-incremental, grep cloud_run/arg-type ===" && python -m mypy backend --no-incremental 2>&1 |
grep -iE "cloud_run|arg-type|job_path" | head; echo "exit: done"; echo "=== mypy -m
backend.compute.cloud_run ===" && python -m mypy -m backend.compute.cloud_run 2>&1 | head -10
[/external_agent_tool_call]
```

38. assistant (2026-07-02T06:11:33.117Z)

```
[external_agent_tool_result]
=== mypy backend --no-incremental, grep cloud_run/arg-type ===
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient" has
incompatible type "str | None"; expected "str"  [arg-type]
exit: done
=== mypy -m backend.compute.cloud_run ===
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient" has
incompatible type "str | None"; expected "str"  [arg-type]
Found 1 error in 1 file (checked 1 source file)
[/external_agent_tool_result]
```

39. assistant (2026-07-02T06:11:33.117Z)

Confirmed ? the earlier pass was a stale `.mypy_cache` artifact. With `--no-incremental` the error reproduces at exactly **line 74** as described:

```
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path"... incompatible type "str |
None"; expected "str"  [arg-type]
```

Now I'll make the fix. I'll guard `project` at the top of `run_job` ? **before** the lazy SDK import, so the guard is exercisable in CI (where `google-cloud-run` is absent) and narrows `Optional[str]` ? `str` for mypy.

40. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py
[/external_agent_tool_call]

41. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

42. assistant (2026-07-02T06:11:33.117Z)

Now the test for the missing-project case. I'll add it to the compute test file, near the other dispatch tests:

43. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py
[/external_agent_tool_call]

44. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

45. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py
[/external_agent_tool_call]

46. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

47. assistant (2026-07-02T06:11:33.117Z)

Now clean up the probe file and run the full verification ? ruff, mypy (no-incremental, SDK installed), and the compute tests:

48. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_call: Bash]
description: Run ruff, mypy, and compute tests
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && rm -f scratch/_mypy_probe.py && echo "=== RUFF (repo-wide, CI lane) ===" && ruff check . 2>&1 | tail -15 && echo "=== MYPY backend (no-incremental, SDK installed) ===" && python -m mypy backend training --no-incremental 2>&1 | tail -8 && echo "=== PYTEST compute backend ===" && python -m pytest tests/test_compute_backend.py -q 2>&1 | tail -25
[/external_agent_tool_call]

49. assistant (2026-07-02T06:11:33.117Z)

[external_agent_tool_result]

```
=== RUFF (repo-wide, CI lane) ===
help: Rewrite `bold` as a `def`

E731 Do not assign a `lambda` expression, use a `def`
  --> artifacts\landing\make_og.py:24:1
   |
23 | bold = lambda s: font("segoeuib.ttf", s)
24 | reg = lambda s: font("segoeui.ttf", s)
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
25 |
26 | M = 80  # margin
   |
help: Rewrite `reg` as a `def`

Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
=== MYPY backend (no-incremental, SDK installed) ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 136 source files
=== PYTEST compute backend ===
............                                                             [100%]
12 passed in 5.32s
[/external_agent_tool_result]
```

50. assistant (2026-07-02T06:11:33.117Z)

mypy now reports **`Success: no issues found in 136 source files`** (the line-74 `arg-type` error is gone), and all **12 compute tests pass** (including the new `test_real_client_rejects_missing_project`).

The 2 ruff errors are in `artifacts/landing/make_og.py` ? a pre-existing, untracked file (`?? artifacts/` in git status) unrelated to my change. Let me confirm my two touched files are ruff-clean:

51. assistant (2026-07-02T06:11:33.117Z)

```
[external_agent_tool_call: Bash]
description: Lint touched files and confirm scope of changes
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== ruff on touched
files ===" && ruff check backend/compute/cloud_run.py tests/test_compute_backend.py && echo "ALL
CLEAN" && echo "=== confirm artifacts is untracked (not my change) ===" && git status --short
backend/ tests/ && echo "=== diff stat of my change ===" && git diff --stat
[/external_agent_tool_call]
```

52. assistant (2026-07-02T06:11:33.117Z)

```
[external_agent_tool_result]
=== ruff on touched files ===
All checks passed!
ALL CLEAN
=== confirm artifacts is untracked (not my change) ===
 M backend/compute/cloud_run.py
 M tests/test_compute_backend.py
=== diff stat of my change ===
warning: in the working copy of 'backend/compute/cloud_run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
```

```
next time Git touches it
 backend/compute/cloud_run.py  |  8 ++++++++
 tests/test_compute_backend.py | 17 ++++++++++++++++++
 2 files changed, 25 insertions(+)
[/external_agent_tool_result]
```

53. assistant (2026-07-02T06:11:33.118Z)

```
Both touched files are ruff-clean (the 2 ruff errors are in the untracked
`artifacts/landing/make_og.py`, unrelated to this change). The diff is scoped to exactly the two
intended files: +8 in the source, +17 in tests.

Here's the final source change:
```

54. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py
[/external_agent_tool_call]
```

55. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_result]
64          def run_job(self, job_name: str, argv: List[str], *, region: str,
65                      project: Optional[str] = None) -> str:
66              # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
67              # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
68              # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
69              if not project:
70                  raise RuntimeError(
71                      "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
72                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
73                  )
74              try:
75                  from google.cloud import run_v2  # type: ignore[attr-defined]
76              except Exception as exc:  # pragma: no cover - real SDK not present in CI
77                  raise RuntimeError(
78                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
79                      "control plane (it is intentionally NOT a CI/test dependency)."
80                  ) from exc
81              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
82              name = client.job_path(project, region, job_name)
83              overrides = run_v2.RunJobRequest.Overrides(
84
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
85              )
[/external_agent_tool_result]
```

56. assistant (2026-07-02T06:11:33.118Z)

```
Done. Here's what I did and what I verified.
```

The bug, confirmed

```
With `google-cloud-run` installed, mypy flagged exactly the line described:
```
```
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient"
  has incompatible type "str | None"; expected "str"  [arg-type]
```

```
`job_path(project: str, location: str, job: str)` is strictly typed (the package ships
`py.typed`), and `project` arrives as `Optional[str]` from
`os.environ.get("GOOGLE_CLOUD_PROJECT")` in
[get_compute_backend](backend/compute/__init__.py:56). CI stays green only because CI doesn't
install the SDK (mypy then sees `run_v2` as `Any`). It's a real latent bug ? a `None`/unset
project builds a bogus `projects/None/locations/...` path on the real dispatch.

> Note: my first `mypy backend` run passed due to a stale `.mypy_cache`. `--no-incremental`
reproduced it ? worth flagging since incremental cache can mask this class of error.

## The fix
A guard at the top of [`GoogleCloudRunJobClient.run_job`](backend/compute/cloud_run.py:69),
placed **before** the lazy `google-cloud-run` import. This:
- **Narrows** `Optional[str]` ? `str` so `job_path(project, ?)` type-checks.
- **Fails loudly** with a clear message naming `GOOGLE_CLOUD_PROJECT` instead of dispatching a
malformed path (covers both `None` and `""`).
- Is **testable in CI** ? the guard runs before the SDK import, so the missing-project path is
exercisable even though the SDK isn't a test dependency.

Added [`test_real_client_rejects_missing_project`](tests/test_compute_backend.py:81), which
asserts a `RuntimeError` matching `GOOGLE_CLOUD_PROJECT` for both `None` and `""`.

## Verification (all green)
| Check | Command | Result |
|---|---|---|
| Types | `mypy backend training --no-incremental` (SDK installed) | **Success, no issues** (136
files); line-74 error gone |
| Lint | `ruff check backend/compute/cloud_run.py tests/test_compute_backend.py` | **All checks
passed** |
| Tests | `pytest tests/test_compute_backend.py` | **12 passed** (was 11 + my new one) |

The 2 ruff errors in the repo-wide run are in `artifacts/landing/make_og.py` ? an untracked file
(`?? artifacts/`) pre-existing and unrelated to this change.

Diff is +25 lines across the two intended files only. I did **not** commit or open a PR (you
didn't ask, and per project convention). Heads-up: the working tree is on a **detached HEAD** ?
if you want this as a PR, I'd cut a branch from `origin/master` and cherry-pick these two files
in.
```

57. user (2026-07-02T06:11:33.118Z)

```
please make the PRs ensure the tests coverage improve and all CI is green and merge it!
```

58. assistant (2026-07-02T06:11:33.118Z)

```
On it. This is one work item ? one PR, and per our working agreement I can merge my own
verified-green engineering-fix PRs. Let me set up safely: confirm the remote/auth, check where
`origin/master` is, and find the coverage floor + full-suite command.
```

59. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_call: Bash]
description: Check gh auth, remote, and origin/master position
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== gh auth ===" &&
gh auth status 2>&1 | head -5 && echo "=== remote ===" && git remote -v | head -2 && echo "===
fetch ===" && git fetch origin 2>&1 | tail -3 && echo "=== HEAD vs origin/master ===" && git log
--oneline -1 HEAD && git log --oneline -1 origin/master
[/external_agent_tool_call]
```

60. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_call: Bash]
```

description: Find coverage floor and full test command
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== pytest.ini ==="
&& cat pytest.ini 2>/dev/null && echo "=== cov-fail-under / cov refs in CI ===" && grep -rnE
"cov-fail-under|--cov|run_all_tests|pytest" .github/workflows/*.yml | head -30 && echo "===
run_all_tests.py head ===" && head -40 run_all_tests.py 2>/dev/null
[/external_agent_tool_call]

61. assistant (2026-07-02T06:11:33.118Z)

[external_agent_tool_result]
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********
=== remote ===
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
=== fetch ===
=== HEAD vs origin/master ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
[/external_agent_tool_result]

62. assistant (2026-07-02T06:11:33.118Z)

[external_agent_tool_result]
=== pytest.ini ===
[pytest]
When `pytest` is invoked without an explicit path, only collect from tests/. scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.) and should never run as part of the regular test suite.

testpaths = tests
=== cov-fail-under / cov refs in CI ===
78:        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
80:        run: python -m pytest tests/test_import_smoke.py -v
102:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
105:      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
112:        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
fail-under=70
133:        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
140:          python -m pytest tests/test_golden_fixtures.py tests/test_golden_real_fixtures.py
-q
=== run_all_tests.py head ===
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py   # full suite, quiet
    python run_all_tests.py -v   # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes

```
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
[/external_agent_tool_result]
```

63. assistant (2026-07-02T06:11:33.118Z)

```
Key findings:
- **`origin/master` has moved ahead**: my detached HEAD is at `9b37269` (#323) but
`origin/master` is at `d92cc10` (#341) ? and #341 is *serving*-related, which could touch
`cloud_run.py`. I must branch from `origin/master` and re-apply on top of the real base.
- **Coverage floor is `--cov-fail-under=70`**; full suite = `python run_all_tests.py`.

Let me back up my working changes as a patch, then check whether #341 touched my two files:
```

64. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_call: Bash]
description: Backup working changes and check if base moved on my files
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && mkdir -p scratch && git
diff -- backend/compute/cloud_run.py tests/test_compute_backend.py > scratch/my_change.patch &&
echo "patch bytes:" && wc -c < scratch/my_change.patch && echo "=== did #323..#341 touch my
files? ===" && git diff 9b37269 origin/master --stat -- backend/compute/cloud_run.py
tests/test_compute_backend.py && echo "=== (empty above = unchanged) ===" && echo "=== full file
list changed #323..origin/master ===" && git diff 9b37269 origin/master --stat | tail -30
[/external_agent_tool_call]
```

65. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_result]
warning: in the working copy of 'backend/compute/cloud_run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
patch bytes:
2639
=== did #323..#341 touch my files? ===
 backend/compute/cloud_run.py  |  3 +-
 tests/test_compute_backend.py | 80 ++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 82 insertions(+), 1 deletion(-)
=== (empty above = unchanged) ===
```

```
=== full file list changed #323..origin/master ===
 backend/infrastructure/database.py       |  60 +++-
 backend/main.py                          | 365 ++++++++++++++++++---
 backend/storage/__init__.py              |  10 +-
 backend/storage/gdrive_api.py            | 178 ++++++++++
 backend/storage/ref.py                   |   7 +-
 backend/ui/THIRD_PARTY_NOTICES.md        |   2 +-
 backend/ui/assets/index-BcE6YOgS.css     |   1 -
 backend/ui/assets/index-GSXqL6iY.js      |  60 ----
 backend/ui/assets/index-Jp9DK5o0.css     |   1 +
 backend/ui/assets/index-ukuS9wxa.js      |  60 ++++
 backend/ui/index.html                    |   4 +-
 backend/ui/ui_version.json               |   6 +-
 backend/worker/__main__.py               |  22 +-
 deploy/cloudrun/Dockerfile               |  22 +-
 deploy/cloudrun/README.md                |   9 +-
 deploy/gcp/nightly_regression/README.md  | 197 +++++++-----
 docs/COMPUTE_DECOUPLED_SERVING/README.md |   6 +-
 docs/NIGHTLY_ENGINE_REGRESSION.md        |   1 +
 docs/README.md                           |   1 +
 scripts/verify_compute_path.py           | 115 +++++++
 tests/test_api_cloud_dispatch.py         | 528 ++++++++++++++++++++++++++++++
 tests/test_api_endpoints.py              |  87 ++++-
 tests/test_async_jobs.py                 |  61 ++-
 tests/test_compute_backend.py            |  80 +++++
 tests/test_database.py                   |  20 +-
 tests/test_flywheel.py                   | 170 ++++++++++
 tests/test_job_worker_loop.py            |   8 +-
 tests/test_locale_analytics.py           |   6 +-
 tests/test_storage_gdrive_api.py         | 186 ++++++++++
 36 files changed, 2304 insertions(+), 256 deletions(-)
[/external_agent_tool_result]
```

66. assistant (2026-07-02T06:11:33.118Z)

Both my files changed on `origin/master` (cloud_run.py: small 3-line change; test file: +80 lines). I must re-apply against the real base, not stash-pop. Let me see exactly what #341 did to `cloud_run.py` so I re-apply cleanly:

67. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_call: Bash]
description: Inspect origin/master changes to my two files
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== cloud_run.py
change #323..origin/master ===" && git diff 9b37269 origin/master --
backend/compute/cloud_run.py && echo "=== test file new tests (names) added in origin/master
===" && git diff 9b37269 origin/master -- tests/test_compute_backend.py | grep -E "^\+def
test|^\+class " | head -40
[/external_agent_tool_call]
```

68. assistant (2026-07-02T06:11:33.118Z)

```
[external_agent_tool_result]
=== cloud_run.py change #323..origin/master ===
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 86ff71e..01c2848 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -129,7 +129,8 @@ class CloudRunCompute(ComputeBackend):
         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc)
-        return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
```

```
+        return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report,
+                             execution=str(execution or ""))

     # --- storage-backed completion signal (bucket-poll) ---
     def _read_marker(self, done_uri: str) -> Optional[dict]:
=== test file new tests (names) added in origin/master ===
+def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
+def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
+def test_dockerfile_uses_micromamba_not_anaconda_tos():
[/external_agent_tool_result]
```

69. assistant (2026-07-02T06:11:33.118Z)

`cloud_run.py`'s #341 change is in `CloudRunCompute.run_infer` (line ~129) ? a different method than mine (`GoogleCloudRunJobClient.run_job`, ~line 64), so no conflict. The test file added 3 unrelated tests, none named like mine. I'll discard my working-tree edits (backed up) and re-apply cleanly on a fresh branch cut from `origin/master`:

70. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_call: Bash]
description: Discard working edits and branch from origin/master
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && git checkout --
backend/compute/cloud_run.py tests/test_compute_backend.py && git checkout -b fix/cloud-run-
missing-project-guard origin/master 2>&1 | tail -3 && echo "=== branch ===" && git branch
--show-current && echo "=== HEAD (should be #341 d92cc10) ===" && git log --oneline -1 && echo
"=== clean tree on tracked files? ===" && git status --short backend/ tests/
[/external_agent_tool_call]
```

71. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_result]
branch 'fix/cloud-run-missing-project-guard' set up to track 'origin/master'.
Previous HEAD position was 9b37269 docs(backlog): track golden-regression-fixtures DoD remainder
(M4, P0 rename) + optional O1/O2 (#323)
Switched to a new branch 'fix/cloud-run-missing-project-guard'
=== branch ===
fix/cloud-run-missing-project-guard
=== HEAD (should be #341 d92cc10) ===
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
=== clean tree on tracked files? ===
[/external_agent_tool_result]
```

72. assistant (2026-07-02T06:11:33.119Z)

Now on a clean branch at `origin/master` (#341). Let me re-read the current `cloud_run.py` around `run_job` and re-apply my edits against this fresh base:

73. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py
[/external_agent_tool_call]
```

74. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_result]
60      class GoogleCloudRunJobClient(CloudRunJobClient):
61          """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
```

```
63
64          def run_job(self, job_name: str, argv: List[str], *, region: str,
65                      project: Optional[str] = None) -> str:
66              try:
67                  from google.cloud import run_v2  # type: ignore[attr-defined]
68              except Exception as exc:  # pragma: no cover - real SDK not present in CI
69                  raise RuntimeError(
70                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                      "control plane (it is intentionally NOT a CI/test dependency)."
72                  ) from exc
73              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
74              name = client.job_path(project, region, job_name)
75              overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77              )
78              op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79              return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
[/external_agent_tool_result]
```

75. assistant (2026-07-02T06:11:33.119Z)

The `run_job` method is unchanged by #341. Re-applying my two edits against this base:

76. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py
[/external_agent_tool_call]
```

77. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

78. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py
[/external_agent_tool_call]
```

79. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

80. assistant (2026-07-02T06:11:33.119Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py
[/external_agent_tool_call]
```

81. assistant (2026-07-02T06:11:33.119Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

82. assistant (2026-07-02T06:11:33.119Z)

Re-applied cleanly. Now verify on the new base ? ruff + mypy first:

83. assistant (2026-07-02T06:11:33.119Z)

[external_agent_tool_call: Bash]
description: ruff + mypy on new base, check pytest-cov
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== ruff touched
files ===" && ruff check backend/compute/cloud_run.py tests/test_compute_backend.py && echo "===
mypy backend training (no-incremental, SDK installed) ===" && python -m mypy backend training
--no-incremental 2>&1 | tail -3 && echo "=== pytest-cov present? ===" && python -c "import
pytest_cov; print('pytest-cov', pytest_cov.__version__)" 2>&1 | head -2
[/external_agent_tool_call]

84. assistant (2026-07-02T06:11:33.119Z)

[external_agent_tool_result]
=== ruff touched files ===
All checks passed!
=== mypy backend training (no-incremental, SDK installed) ===
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 138 source files
=== pytest-cov present? ===
pytest-cov 7.1.0
[/external_agent_tool_result]

85. assistant (2026-07-02T06:11:33.120Z)

ruff clean, mypy `Success` (138 files), pytest-cov present. Now the full suite with coverage ?
replicating the exact CI command (floor 70). Let me also capture `cloud_run.py`'s own coverage
to prove the improvement:

86. assistant (2026-07-02T06:11:33.120Z)

[external_agent_tool_call: Bash]
description: Run full suite with coverage (CI command)
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && python run_all_tests.py
--cov=backend --cov=training --cov-report=term --cov-fail-under=70 2>&1 | tail -30
[/external_agent_tool_call]

87. assistant (2026-07-02T06:11:33.120Z)

[external_agent_tool_result]
Command running in background with ID: bacrfugc8. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output. You will be notified when
it completes. To check interim output, use Read on that file path.
[/external_agent_tool_result]

88. assistant (2026-07-02T06:11:33.120Z)

Full suite is running in the background. Let me check interim progress:

89. assistant (2026-07-02T06:11:33.120Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output
[/external_agent_tool_call]

90. assistant (2026-07-02T06:11:33.120Z)

[external_agent_tool_result]
<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>
[/external_agent_tool_result]

91. assistant (2026-07-02T06:11:33.120Z)

The suite is running (pytest writes its summary at the end). I'll wait for it to complete before committing ? green local suite is a prerequisite. I'll continue as soon as it finishes.

92. assistant (2026-07-02T06:11:33.120Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output
[/external_agent_tool_call]

93. assistant (2026-07-02T06:11:33.120Z)

[external_agent_tool_result]
<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>
[/external_agent_tool_result]

94. assistant (2026-07-02T06:11:33.120Z)

I'll wait for the completion notification rather than polling.

95. user (2026-07-02T06:11:33.121Z)

<task-notification>
<task-id>bacrfugc8</task-id>
<tool-use-id>toolu_01GJJFS9Gvw3tBUvyaznYVhZ</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output</output-file>
<status>completed</status>
<summary>Background command "Run full suite with coverage (CI command)" completed (exit code 0)</summary>
</task-notification>

96. assistant (2026-07-02T06:11:33.121Z)

Suite completed with exit code 0. Let me read the full output to confirm the result and coverage:

97. assistant (2026-07-02T06:11:33.121Z)

[external_agent_tool_call: Bash]
description: Read full suite output tail
command: cat "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output" | tail -30
[/external_agent_tool_call]

98. assistant (2026-07-02T06:11:33.121Z)

[external_agent_tool_result]

```
backend\storage\local.py                              49      3    94%
backend\storage\ref.py                                48      0   100%
backend\storage\s3.py                                 47      5    89%
backend\tools\__init__.py                              0      0   100%
backend\tools\export.py                               75      2    97%
backend\tools\rally_slicer.py                        105     47    55%
backend\utils\__init__.py                              0      0   100%
backend\utils\app_home.py                             40      0   100%
backend\utils\ffmpeg.py                               24      0   100%
backend\utils\geometry.py                             29      0   100%
backend\utils\hashing.py                               9      0   100%
backend\utils\logging_setup.py                        11      0   100%
backend\utils\paths.py                                33      3    91%
backend\utils\redact.py                               16      0   100%
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\single_instance.py                      43     11    74%
backend\utils\video.py                                26      3    88%
backend\utils\workspace.py                            49      0   100%
backend\worker\__init__.py                             0      0   100%
backend\worker\__main__.py                           304     29    90%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             129      9    93%
------------------------------------------------------------------------
TOTAL                                              10342   1462    86%
Required test coverage of 70% reached. Total coverage: 85.86%
1488 passed, 7 warnings in 168.88s (0:02:48)
========================================================================
 RESULT: PASS - all tests green
========================================================================
[/external_agent_tool_result]
```

99. assistant (2026-07-02T06:11:33.121Z)

Full suite: **1488 passed**, total coverage **85.86%** (floor 70 cleared). Let me confirm `cloud_run.py`'s specific coverage (proving the new guard lines are covered):

100. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_call: Bash]
description: Show cloud_run.py coverage line
command: grep -iE "cloud_run|compute" "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-
Projects-badminton-highlight-
indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output"
[/external_agent_tool_call]
```

101. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_result]
(Bash completed with no output)
[/external_agent_tool_result]
```

102. assistant (2026-07-02T06:11:33.121Z)

Let me run focused coverage on `cloud_run.py` to demonstrate the new guard lines are covered:

103. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_call: Bash]
description: Focused coverage on cloud_run.py
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && python -m pytest
tests/test_compute_backend.py -q --cov=backend.compute.cloud_run --cov-report=term-missing 2>&1
| grep -iE "cloud_run|passed|TOTAL|Name "
[/external_agent_tool_call]
```

104. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_result]
Name                            Stmts   Miss  Cover   Missing
backend\compute\cloud_run.py       74     11    85%   74-87, 134-136, 162-164
TOTAL                              74     11    85%
15 passed in 8.19s
[/external_agent_tool_result]
```

105. assistant (2026-07-02T06:11:33.121Z)

`cloud_run.py` is at **85%** ? the new guard lines are covered (the missing 74-87 are the `# pragma: no cover` real-SDK path), and the compute suite grew to 15 tests. All local gates are green:

- **ruff**: clean
- **mypy backend training** (SDK installed, no-incremental): `Success` ? the latent `arg-type` error is gone
- **full suite**: 1488 passed, **85.86%** coverage (floor 70 cleared)

Now commit (explicit paths, after re-verifying HEAD), push, and open the PR:

106. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_call: Bash]
description: Stage explicit paths and commit
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== HEAD re-verify
(mine) ===" && git branch --show-current && git log --oneline -1 && echo "=== stage explicit
paths ===" && git add backend/compute/cloud_run.py tests/test_compute_backend.py && git status
--short && cat > scratch/commit_msg.txt <<'EOF'
fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT

run_job passed `project` (Optional[str], sourced from the GOOGLE_CLOUD_PROJECT
env in get_compute_backend, which can be None) straight to
`JobsClient.job_path(project: str, ...)`. mypy flags this as an arg-type error
once google-cloud-run is installed; CI stays green only because the SDK is
intentionally NOT a CI/test dependency. It is a real latent bug: a None/empty
project would build a bogus "projects/None/locations/..." path on the real
Cloud Run dispatch.

Validate `project` BEFORE the lazy SDK import and raise a clear RuntimeError
naming GOOGLE_CLOUD_PROJECT. This both narrows Optional[str] -> str for
job_path (mypy clean) and fails loudly on a misconfigured control plane.
Running before the import means the missing-project case is unit-testable in
CI even though the SDK is absent there.

Add test_real_client_rejects_missing_project covering project=None and "".

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git commit -F scratch/commit_msg.txt 2>&1 | tail -4
[/external_agent_tool_call]
```

107. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_result]
=== HEAD re-verify (mine) ===
fix/cloud-run-missing-project-guard
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
=== stage explicit paths ===
M  backend/compute/cloud_run.py
M  tests/test_compute_backend.py
?? artifacts/
```

```
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
[fix/cloud-run-missing-project-guard fc8fb14] fix(compute): guard GoogleCloudRunJobClient
against an unset GOOGLE_CLOUD_PROJECT
 2 files changed, 25 insertions(+)
[/external_agent_tool_result]
```

108. assistant (2026-07-02T06:11:33.121Z)

```
Committed `fc8fb14`. Verify the branch contains only my commit vs `origin/master`, then push and
open the PR:
```

109. assistant (2026-07-02T06:11:33.121Z)

```
[external_agent_tool_call: Bash]
description: Verify branch isolation and push
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== commits on branch
vs origin/master (must be ONLY mine) ===" && git log --oneline origin/master..HEAD && echo "===
push ===" && git push -u origin fix/cloud-run-missing-project-guard 2>&1 | tail -6
[/external_agent_tool_call]
```

110. assistant (2026-07-02T06:11:33.122Z)

```
[external_agent_tool_result]
=== commits on branch vs origin/master (must be ONLY mine) ===
fc8fb14 fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT
=== push ===
remote: Create a pull request for 'fix/cloud-run-missing-project-guard' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/cloud-run-
missing-project-guard
remote:
branch 'fix/cloud-run-missing-project-guard' set up to track 'origin/fix/cloud-run-missing-
project-guard'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/cloud-run-missing-project-guard -> fix/cloud-run-missing-project-guard
[/external_agent_tool_result]
```

111. assistant (2026-07-02T06:11:33.122Z)

```
Branch isolation confirmed (only my commit). Now create the PR:
```

112. assistant (2026-07-02T06:11:33.122Z)

```
[external_agent_tool_call: Bash]
description: Create the PR
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && cat > scratch/pr_body.md
<<'EOF'
```

What

```
`GoogleCloudRunJobClient.run_job` passed `project` ? an `Optional[str]` sourced from the
`GOOGLE_CLOUD_PROJECT` env in `get_compute_backend` (it's `None` when unset) ? straight to
`JobsClient.job_path(project: str, ...)`.

- **mypy flags it** as `arg-type` (`"str | None"` vs expected `"str"`) *once `google-cloud-run`
is installed*. CI stays green today only because the SDK is intentionally **not** a CI/test
dependency ? so the broad `mypy backend` lane never sees the real `job_path` signature. It's a
real latent bug, not just a typing nit: a `None`/empty project builds a bogus
`projects/None/locations/...` path on the real Cloud Run dispatch.
```

Fix

```
Validate `project` **before** the lazy `google-cloud-run` import and raise a clear
`RuntimeError` naming `GOOGLE_CLOUD_PROJECT`. This:
```

- **narrows** `Optional[str]` ? `str` for `job_path` (mypy clean), and
- **fails loudly** on a misconfigured control plane instead of dispatching a malformed path.

Placing the guard before the import keeps the missing-project case **unit-testable in CI** even though the SDK is absent there.

Test (coverage ?)

Adds `test_real_client_rejects_missing_project` covering `project=None` and `""`. The compute suite grows 14 ? 15 tests; `backend/compute/cloud_run.py` stays at 85% (the only uncovered lines are the `# pragma: no cover` real-SDK path).

Verification (local, all green)

- `ruff check` on touched files ? clean
- `mypy backend training --no-incremental` **with `google-cloud-run` installed** ? `Success: no issues found in 138 source files` (reproduced the error before the fix at `cloud_run.py:74`, gone after)
- `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? **1488 passed**, total coverage **85.86%**

? Generated with [Claude Code](https://claude.com/claude-code)
EOF
gh pr create --base master --head fix/cloud-run-missing-project-guard --title "fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT" --body-file scratch/pr_body.md 2>&1 | tail -5
[/external_agent_tool_call]

113. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_result]
https://github.com/Khelsutra/badminton-highlight-indexer/pull/342
[/external_agent_tool_result]

114. assistant (2026-07-02T06:11:33.122Z)

PR created: [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342). Now let me watch CI. First, see what checks are registered:

115. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_call: Bash]
description: Check initial CI status and mergeability
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && sleep 8; gh pr checks 342 2>&1 | head -20; echo "=== mergeable state ==="; gh pr view 342 --json mergeable,mergeStateStatus,reviewDecision -q '{mergeable: .mergeable, state: .mergeStateStatus, review: .reviewDecision}'
[/external_agent_tool_call]

116. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_result]
Built wheel smoke (install + launch)   pending 0   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859922
Dependency license scan (no copyleft in runtime deps)   pending 0   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859677
Full test suite (offline, mocked)   pending 0   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859930
Golden fixtures cross-OS (macos-latest) pending 0   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794860679
Golden fixtures cross-OS (windows-latest)   pending 0   https://github.com/Khelsutra/badminton-highlight-

indexer/actions/runs/27976307759/job/82794859801
Leak scan (PII / attribution guard) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859809
Syntax sweep + import smoke (no GPU stack) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859736
Ruff lint pass 9s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859797
Golden fixtures cross-OS (ubuntu-latest) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859796
Mypy (lenient baseline) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859772
=== mergeable state ===
{"mergeable":"MERGEABLE","review":"","state":"UNSTABLE"}
[/external_agent_tool_result]

117. assistant (2026-07-02T06:11:33.122Z)

CI is running (Ruff already ?). I'll watch the checks in the background ? `gh pr checks --watch` exits when all complete (non-zero if any fail):

118. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_call: Bash]
description: Watch PR CI checks until completion
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && gh pr checks 342 --watch --interval 30 2>&1 | tail -20; echo "=== watch exit: $? ==="
[/external_agent_tool_call]

119. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_result]
Command running in background with ID: bncmmz9h2. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bncmmz9h2.output. You will be notified when it completes. To check interim output, use Read on that file path.
[/external_agent_tool_result]

120. assistant (2026-07-02T06:11:33.122Z)

I'll wait for CI to complete (watching in the background) and then merge once it's verified green.

121. user (2026-07-02T06:11:33.122Z)

<task-notification>
<task-id>bncmmz9h2</task-id>
<tool-use-id>toolu_01Eut5gQqbGptTwtoKYNcbRD</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bncmmz9h2.output</output-file>
<status>completed</status>
<summary>Background command "Watch PR CI checks until completion" completed (exit code 0)</summary>
</task-notification>

122. assistant (2026-07-02T06:11:33.122Z)

The watch exited 0 (all checks passed). Let me confirm the full check results and mergeability before merging:

123. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_call: Bash]

description: Confirm all checks passed and mergeability
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && gh pr checks 342 2>&1 |
head -20 && echo "=== mergeable ===" && gh pr view 342 --json mergeable,mergeStateStatus -q
'{mergeable: .mergeable, state: .mergeStateStatus}'
[/external_agent_tool_call]

124. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_result]
Built wheel smoke (install + launch) pass 1m12s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794859922
Dependency license scan (no copyleft in runtime deps) pass 38s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859677
Full test suite (offline, mocked) pass 2m36s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794859930
Golden fixtures cross-OS (macos-latest) pass 28s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794860679
Golden fixtures cross-OS (ubuntu-latest) pass 21s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859796
Golden fixtures cross-OS (windows-latest) pass 1m16s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859801
Leak scan (PII / attribution guard) pass 9s https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794859809
Mypy (lenient baseline) pass 29s https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859772
Ruff lint pass 9s https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859797
Syntax sweep + import smoke (no GPU stack) pass 24s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859736
=== mergeable ===
{"mergeable":"MERGEABLE","state":"CLEAN"}
[/external_agent_tool_result]

125. assistant (2026-07-02T06:11:33.122Z)

All 10 checks pass; state is **CLEAN**. Merging now (squash, matching the repo's `(#NNN)`
convention) and deleting the branch:

126. assistant (2026-07-02T06:11:33.122Z)

[external_agent_tool_call: Bash]
description: Squash-merge the PR and delete branch
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && gh pr merge 342 --squash
--delete-branch 2>&1 | tail -8 && echo "=== PR state ===" && gh pr view 342 --json
state,mergedAt -q '{state: .state, mergedAt: .mergedAt}'
[/external_agent_tool_call]

127. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_result]
failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'

=== PR state ===
{"mergedAt":"2026-06-22T18:55:18Z","state":"MERGED"}
[/external_agent_tool_result]

128. assistant (2026-07-02T06:11:33.123Z)

PR **#342 is MERGED** (`mergedAt` set). The `git` error was only gh's post-merge attempt to

switch the local checkout to `master`, which is held by another worktree
(`.claude/worktrees/tier1-windowing`) ? harmless. Let me confirm the remote branch cleanup and
the merge landed on `origin/master`:

129. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_call: Bash]
description: Verify remote branch deletion and merge on origin/master
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && git fetch origin --prune
2>&1 | tail -3 && echo "=== remote branch gone? (empty = deleted) ===" && git ls-remote --heads
origin fix/cloud-run-missing-project-guard && echo "=== origin/master tip (should be my squash
merge #342) ===" && git log --oneline -2 origin/master
[/external_agent_tool_call]

130. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_result]
 - [deleted]   (none)   -> origin/serving/m11-flywheel-skeleton
 - [deleted]   (none)   -> origin/serving/m12-gdrive-api
   d92cc10..688d28f master   -> origin/master
=== remote branch gone? (empty = deleted) ===
=== origin/master tip (should be my squash merge #342) ===
688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
[/external_agent_tool_result]

131. assistant (2026-07-02T06:11:33.123Z)

Confirmed merged: `origin/master` advanced to `688d28f fix(compute): guard
GoogleCloudRunJobClient... (#342)`, and the remote branch was deleted. Let me tidy the local
state ? return to a detached HEAD on the new `origin/master` (matching how the session started,
since `master` is held by another worktree), delete the merged local branch, and clear my
scratch files:

132. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_call: Bash]
description: Detach to origin/master, delete merged branch, clean scratch
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && git checkout --detach
origin/master 2>&1 | tail -2 && git branch -D fix/cloud-run-missing-project-guard 2>&1 | tail -1
&& rm -f scratch/commit_msg.txt scratch/pr_body.md scratch/my_change.patch && echo "=== final
state ===" && git log --oneline -1 && git status --short backend/ tests/
[/external_agent_tool_call]

133. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_result]
HEAD is now at 688d28f fix(compute): guard GoogleCloudRunJobClient against an unset
GOOGLE_CLOUD_PROJECT (#342)
Deleted branch fix/cloud-run-missing-project-guard (was fc8fb14).
=== final state ===
688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
[/external_agent_tool_result]

134. assistant (2026-07-02T06:11:33.123Z)

Done ? shipped and merged. ?

Summary

**PR [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342) ? merged to
`master`** as `688d28f`.

What it fixes
`GoogleCloudRunJobClient.run_job` passed `project` (`Optional[str]`, from `GOOGLE_CLOUD_PROJECT` which can be `None`) straight to `JobsClient.job_path(project: str, ?)`. A guard now validates `project` **before** the lazy SDK import and raises a clear `RuntimeError` naming `GOOGLE_CLOUD_PROJECT` ? which both narrows `Optional[str]` ? `str` for mypy and fails loudly instead of dispatching a bogus `projects/None/?` path.

Verified (not rubber-stamped)
- **Reproduced the latent bug first**: with `google-cloud-run` installed, `mypy` flagged `cloud_run.py:74` (`"str | None"` vs expected `"str"`); confirmed it was a stale-cache false-pass via `--no-incremental`.
- **mypy** `backend training` (SDK installed): `Success, 138 files` ? error gone.
- **ruff**: clean on touched files.
- **Full suite locally**: `1488 passed`, total coverage **85.86%** (floor 70).
- **Coverage ?**: new `test_real_client_rejects_missing_project` (covers `None` and `""`); compute suite 14 ? 15 tests; new guard lines are covered.
- **CI: all 10 checks green** (CLEAN) ? including the full suite, mypy, ruff, cross-OS golden fixtures, wheel smoke, license/leak scans.

Process notes
- The branch base had moved (`origin/master` was at #341, ahead of the session's detached HEAD at #323, and #341 had touched both files). I re-cut the branch from `origin/master` and re-applied cleanly, verified `origin/master..HEAD` contained only my commit, and staged explicit paths ? per the shared-checkout branching-safety convention.
- Squash-merged + remote branch deleted. The one `git` error during merge was only gh trying to switch the local checkout to `master`, which is held by the `tier1-windowing` worktree ? harmless; the merge succeeded. Local checkout returned to a detached HEAD on the new `origin/master`.

This fell under the "merge my own verified-green engineering-fix PRs" working agreement (a type-safety/correctness fix, not product/strategy/licensing), so I merged it. Want me to refresh the session handoff / current-state memory to reflect this landing?

135. user (2026-07-02T06:11:33.123Z)

oh yes please do update the memories, handoffs and docs with the completed work!

136. assistant (2026-07-02T06:11:33.123Z)

Let me read the current memory/handoff files so I update them accurately (not duplicate or overwrite live state):

137. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

138. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_result]
<system-reminder>[Truncated: PARTIAL view ? showing lines 1-47 of 189 total (84853 tokens, cap 25000). Call Read with offset=48 limit=47 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
```

```
8       ---
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      > **Scope of this file = LIVE only:** the current handoff + *today's* checkpoints + open
PRs/decisions. Keep it read-first-short (?1 page). Older checkpoints are archived in [[current-
state-archive]] ? when today's checkpoints age out (next clean-slate day), move them there,
don't let them pile up here.
13
14      **Checkpoint:** 2026-06-23 (**CLOUD TOGGLE LIT + VALIDATION METHOD + LIVE E2E PROVEN +
user guide + droplet full-suite**). master `688d28f`+. Owner: "light up the cloud compute toggle
with Cloud Run + a good validation method to ensure the intended path was taken" ? then "run the
test on the droplet with a full install" ? "write a doc to help users set up + create reels."
ALL DONE:
15      - **? Validation method ? engine [#341](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/341) MERGED (`d92cc10`):** compute-path PROVENANCE ? the job result
carries `compute={path:"in_process"|"cloud_run"|"not_run", requested, execution, job, region,
outputs_uri}` (the cloud `execution` = the real Cloud Run execution name, GCP-verifiable).
`ComputeResult.execution` threaded via `run_infer`; honest failure records (path=cloud_run only
when it actually dispatched+ran, else not_run); `[COMPUTE]` audit log;
**`scripts/verify_compute_path.py`** asserts the path + cross-checks the execution in GCP.
**Also forwards `indexing.skip_ai_handoff` to the cloud job** so the picker's cloud path yields
rallies without a Gemini key. Adversarial review caught 2 honesty bugs (false "report carries
it" claim; misleading log on a never-dispatched reject) ? fixed. Suite 1480?.
16      - **? LIVE E2E PROVEN ($0 after teardown):** rebuilt the Cloud Run L4 image + Job; ran a
local **cloud-serving** engine (`deployment.profile=cloud-serving` + `skip_ai_handoff=true`
override + `CLOUD_RUN_JOB/REGION=asia-southeast1/PROJECT` + `storage.output_root`) ?
`/api/capabilities` reported **`cloud_available:true`** (toggle lit). `verify_compute_path.py
--compute cloud` ? **`path=cloud_run`, execution `rally-worker-qbcqb`** (GCP cross-check:
succeededCount=1) ? **22 rallies** (skip_ai_handoff forward works). `--compute local` ?
**`path=in_process`** ?. Both directions provably correct. Proof logs in
`artifacts/cloudrun_lightup/`. Job+image+AR repo+prefix deleted.
17      - **? Verify-script Windows bug ? [#344](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/344) OPEN:** `--verify-gcp` spawned bare `gcloud` (Windows =
`gcloud.cmd`, WinError 2) ? resolve via `shutil.which`. Found during the live run (path
assertion passed first).
18      - **? Sibling merged [#342](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/342)** (`688d28f`) ? the `GOOGLE_CLOUD_PROJECT` Optional[str] guard I spawned as a
chip; that latent mypy nit is fixed.
19      - **? Droplet FULL-install suite GREEN:** pushed master?`root@168.144.126.176:~/bhi-
fulltest`, fresh venv + `pip install -e .[dev,auth,storage-gcs,storage-s3]` + torch CPU +
ffmpeg/cv2 libs ? **1461 passed, 12 skipped** (the skips = the private `obsreport` git+ssh dep,
not installed). Droplet restored (dir removed, 1.6G freed).
20      - **? User guide ? [#343](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/343) OPEN:** new **`docs/CREATING_REELS.md`** (end-user: prereqs?`indexer
--app`?wizard?reel via web UI AND the API chain capabilities?upload/...
[/external_agent_tool_result]
```

139. assistant (2026-07-02T06:11:33.123Z)

The `current-state.md` top checkpoint (2026-06-23) already records #342 at line 18 ? a sibling session noted it (it spawned the original chip; this session implemented + merged it). Let me read the handoff and check whether any doc references this latent bug before deciding what to touch:

140. assistant (2026-07-02T06:11:33.123Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\session-handoff.md
[/external_agent_tool_call]
```

141. assistant (2026-07-02T06:11:33.123Z)

[external_agent_tool_result]
1 ---
2 name: session-handoff
3 description: "Living \"you are here\" handoff ? current state, next steps, open decisions, ramp-up kit. AUTO-REFRESHED at meaningful checkpoints (PR merged, key decision, phase shift, session wrap). \"update the handoff\" forces a full rewrite."
4 metadata:
5 node_type: memory
6 type: project
7 originSessionId: 45ea3a91-654d-4972-9cd3-65f35db54caf
8 ---
9
10 # Session handoff ? badminton-highlight-indexer (rename PENDING) + siblings: sports-data-collector, rally-annotator, sports-obsreport
11
12 **Last updated:** 2026-06-22 (**M11+M12 serving-code session** ? owner: "build 3 then 4 then we pair on the alpha (2)". **Both Claude-doable engine items DONE + merged:** **M11** data-flywheel skeleton [#336](https://github.com/Khelsutra/badminton-highlight-indexer/pull/336) (`334ad0f` ? default-OFF + consent FAKED-DENY, inert; reuses workspace/eval/promotion; NO migration) + **M12** `gdrive-api://` OAuth StorageBackend [#338](https://github.com/Khelsutra/badminton-highlight-indexer/pull/338) (`819e395` ? idempotent put, lazy Google imports, injected-fake tests; live per-user OAuth owner-gated). Both adversarially reviewed, full-suite+cross-OS green, merged on green, in an isolated worktree. **? NEXT = pair with owner on the ALPHA GO-LIVE (option 2)** ? owner brings CF dashboard + box + rotated Gemini key; the counsel ToS/DPA inputs (the 7-pt checklist) gate M11 *live* + M9-consent. ? schema ladder drifted under me this session: v7=`locale`, v8=compute-picker ? the future M9-consent migration is **v9** (check `PRAGMA user_version` before authoring). Prior same-day: **maintenance / green-PR merge-sweep session** ? owner asked to merge the green PRs + finish loose ends. 6 PRs now on master: worker dated-label fix **#319**, nightly Cloud-Run scheduler **#320**, DATA_IN_GCS map **#316**, doc **#321**, gate-A litmus re-ground **#322**, golden-fixtures backlog **#323**. engine `origin/master` = `9b37269`, suite **1428?**, **0 open engine PRs**; docs-archive had no terminal candidate (owner-gated). Full per-PR detail = [[current-state]] top. ? ~10 concurrent worktrees/sessions ? owner is trimming concurrency before new work. The serving/alpha plan below is UNCHANGED. Prior **2026-06-21 ALPHA-SHAREABLE session** ? Claude's half of "make it UI-driven + alpha-shareable" is **DONE**: 3 PRs merged ? engine **#294** exposure-safety boot posture + khelsutra **#64** SPA `VITE_API_BASE` + **#65** deploy kit. **KEY CORRECTION:** B-P2 (SPA ingest screen) was ALREADY shipped (#56) ? alpha-shareable was a DEPLOY/AUTH task, not an SPA build. Owner chose the **SPLIT** topology (SPA on CF Pages + `api.*` cloudflared tunnel + CF Access). Remaining = OWNER stand-up via `khelsutra/deploy/DEPLOY_ALPHA.md`; then engine M11/M12. Full record [[ui-driven-cloud-serving-gap]]). Prior: HUGE serving session ? M1?M9-auth shipped (#279?#291) + real M7 L4 run. **Repo is under the `Khelsutra` GitHub org**.
13
14 > **Read order:** this file ? [[current-state]] top (dense per-checkpoint detail) ? `docs/NEXT_STEPS.md` (the prioritized cross-track plan, refreshed 2026-06-21) ? `docs/DOC_STATUS.md` (doc lifecycle/hygiene).
15
16 ## ? You are here (2026-06-22)
17 **(2026-06-22 M11+M12)** engine `origin/master` = **`819e395`** (past #336 M11 + #338 M12; also sibling #333 M6?/api/process, #334 Cloud-Run env, #6d3d054 SPA compute-picker) · khelsutra `origin/master` = `ba38f15` (past #96 Privacy Policy, #100). **The two Claude-doable serving-code items are DONE** (M11 flywheel skeleton #336 + M12 gdrive-api #338 ? both default-OFF/inert, adversarially reviewed, merged on green). **Claude's serving + alpha-shareable code is now complete.** **?? THE ONLY REMAINING ALPHA STEP IS OWNER-DRIVEN: pair on the go-live** (cloudflared tunnel + CF Pages + CF Access + Gemini-key rotation per `khelsutra/deploy/DEPLOY_ALPHA.md`) ? Claude guides, owner drives the CF dashboard/box. M11 going *live* + M9-consent need the counsel ToS/DPA inputs (the 7-pt che...
[/external_agent_tool_result]

142. assistant (2026-07-02T06:11:33.123Z)

The handoff is at 2026-06-22; `current-state` already records #342. Let me check whether any doc references `GOOGLE_CLOUD_PROJECT` / the cloud-run env setup, so a doc note would land somewhere natural rather than being aspirational noise:

143. assistant (2026-07-02T06:11:33.123Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"GOOGLE_CLOUD_PROJECT","output_mode":"files_with_matches"}
[/external_agent_tool_call]
```

144. assistant (2026-07-02T06:11:33.123Z)

```
[external_agent_tool_result]
Found 8 files
tests\test_compute_backend.py
backend\compute\cloud_run.py
tests\test_api_cloud_dispatch.py
deploy\cloudrun\README.md
backend\main.py
backend\compute\__init__.py
artifacts\cloudrun_e2e_validation\RUN_TRANSCRIPT.md
backend\config\startup.py
[/external_agent_tool_result]
```

145. assistant (2026-07-02T06:11:33.124Z)

`deploy/cloudrun/README.md` is the natural home for an env-setup note. Let me see its `GOOGLE_CLOUD_PROJECT` context:

146. assistant (2026-07-02T06:11:33.124Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"GOOGLE_CLOUD_PROJECT|CLOUD_RUN_JOB|CLOUD_RUN_REGION|project","path":"deploy\\
cloudrun\\README.md","output_mode":"content","-n":true,"-C":3}
[/external_agent_tool_call]
```

147. assistant (2026-07-02T06:11:33.124Z)

```
[external_agent_tool_result]
9-> cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does **not**
build this image.
10-
11-## 0. Prerequisites (owner)
12:- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13-- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore
? see
14-  `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15-- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16-
17-## 1. Build + push the image (Artifact Registry)
18-```bash
19:PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20-gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21-gcloud builds submit --tag $REGION-docker.pkg.dev\$PROJECT\$REPO\$IMG:latest -f
deploy/cloudrun/Dockerfile .
22-```
--
37-`compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute` from
env:
38-```bash
39-export DEPLOYMENT_PROFILE=cloud-serving
40:export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION  GOOGLE_CLOUD_PROJECT=$PROJECT
41-# A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job instead
of
42-# running in-process; the container runs the SAME worker INLINE (compute_target forced
```

local_gpu).
43-python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
[/external_agent_tool_result]

148. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\README.md
[/external_agent_tool_call]

149. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_result]
34
35   ## 3. Wire the dispatcher (control plane)
36   The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37   `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38   ```bash
39   export DEPLOYMENT_PROFILE=cloud-serving
40   export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41   # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42   # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43   python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44   ```
45
46   ## 4. M7 acceptance (the owner runlog)
47   Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48   `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49   rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50   exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52   ## Gotchas
53   - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54   + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55   - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
[/external_agent_tool_result]

150. assistant (2026-07-02T06:11:33.124Z)

`deploy/cloudrun/README.md` is the natural operator home ? its "Gotchas" section should note the new fail-fast. I'll do all three: enrich the `current-state` #342 line, add a handoff note, and ship a small doc PR for the README gotcha.

First, the memory updates:

151. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md
[/external_agent_tool_call]

152. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_result: error]
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
[/external_agent_tool_result]

153. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

154. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_result]
14 **Checkpoint:** 2026-06-23 (**CLOUD TOGGLE LIT + VALIDATION METHOD + LIVE E2E PROVEN + user guide + droplet full-suite**). master `688d28f`+. Owner: "light up the cloud compute toggle with Cloud Run + a good validation method to ensure the intended path was taken" ? then "run the test on the droplet with a full install" ? "write a doc to help users set up + create reels." ALL DONE:
15 - **? Validation method ? engine [#341](https://github.com/Khelsutra/badminton-highlight-indexer/pull/341) MERGED (`d92cc10`):** compute-path PROVENANCE ? the job result carries `compute={path:"in_process"|"cloud_run"|"not_run", requested, execution, job, region, outputs_uri}` (the cloud `execution` = the real Cloud Run execution name, GCP-verifiable). `ComputeResult.execution` threaded via `run_infer`; honest failure records (path=cloud_run only when it actually dispatched+ran, else not_run); `[COMPUTE]` audit log; **`scripts/verify_compute_path.py`** asserts the path + cross-checks the execution in GCP. **Also forwards `indexing.skip_ai_handoff` to the cloud job** so the picker's cloud path yields rallies without a Gemini key. Adversarial review caught 2 honesty bugs (false "report carries it" claim; misleading log on a never-dispatched reject) ? fixed. Suite 1480?.
16 - **? LIVE E2E PROVEN ($0 after teardown):** rebuilt the Cloud Run L4 image + Job; ran a local **cloud-serving** engine (`deployment.profile=cloud-serving` + `skip_ai_handoff=true` override + `CLOUD_RUN_JOB/REGION=asia-southeast1/PROJECT` + `storage.output_root`) ? `/api/capabilities` reported **`cloud_available:true`** (toggle lit). `verify_compute_path.py --compute cloud` ? **`path=cloud_run`, execution `rally-worker-qbcqb`** (GCP cross-check: succeededCount=1) ? **22 rallies** (skip_ai_handoff forward works). `--compute local` ? **`path=in_process`** ?. Both directions provably correct. Proof logs in `artifacts/cloudrun_lightup/`. Job+image+AR repo+prefix deleted.
17 - **? Verify-script Windows bug ? [#344](https://github.com/Khelsutra/badminton-highlight-indexer/pull/344) OPEN:** `--verify-gcp` spawned bare `gcloud` (Windows = `gcloud.cmd`, WinError 2) ? resolve via `shutil.which`. Found during the live run (path assertion passed first).
18 - **? Sibling merged [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342)** (`688d28f`) ? the `GOOGLE_CLOUD_PROJECT` Optional[str] guard I spawned as a chip; that latent mypy nit is fixed.
19 - **? Droplet FULL-install suite GREEN:** pushed master?`root@168.144.126.176:~/bhi-fulltest`, fresh venv + `pip install -e .[dev,auth,storage-gcs,storage-s3]` + torch CPU + ffmpeg/cv2 libs ? **1461 passed, 12 skipped** (the skips = the private `obsreport` git+ssh dep, not installed). Droplet restored (dir removed, 1.6G freed).
20 - **? User guide ? [#343](https://github.com/Khelsutra/badminton-highlight-indexer/pull/343) OPEN:** new **`docs/CREATING_REELS.md`** (end-user: prereqs?`indexer --app`?wizard?reel via web UI AND the API chain capabilities?upload/gs://?process[202+poll]?query?compile[202+poll]?download; +cloud toggle +troubleshooting). Researched via a verify-then-write workflow (every command/endpoint/flag checked vs the shipped engine; LIGHT-tier-honest). Indexed in docs/README.md + DOC_STATUS.md §3d.
21 - **? NEXT:** merge #343+#344 on green; clean worktrees (bhi-computeval/bhi-userdoc/bhi-verifyfix). Owner follow-up to make the toggle live on the ALPHA = deploy a persistent Cloud Run Job + flip the droplet engine to cloud-serving (a deploy decision, not code). [[cloudrun-gpu-serving-gotchas]] [[gcp-vm-always-delete]] [[working-agreement-merging]]
22
[/external_agent_tool_result]

155. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

156. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

157. assistant (2026-07-02T06:11:33.124Z)

Now add a concise note to the handoff's "You are here":

158. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md
[/external_agent_tool_call]

159. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

160. assistant (2026-07-02T06:11:33.124Z)

Memory + handoff updated. Now the doc PR. Let me fetch (siblings may have moved master), branch from `origin/master`, and add the operator gotcha:

161. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_call: Bash]
description: Branch from origin/master for the doc PR
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && git fetch origin 2>&1 | tail -2 && echo "=== origin/master ===" && git log --oneline -1 origin/master && git checkout -b docs/cloudrun-require-project origin/master 2>&1 | tail -2 && git branch --show-current
[/external_agent_tool_call]

162. assistant (2026-07-02T06:11:33.124Z)

[external_agent_tool_result]
From https://github.com/Khelsutra/badminton-highlight-indexer
   688d28f..239c8eb master   -> origin/master
=== origin/master ===
239c8eb fix(scripts): resolve gcloud via PATH in verify_compute_path so --verify-gcp works on Windows (#344)
Previous HEAD position was 688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
Switched to a new branch 'docs/cloudrun-require-project'
docs/cloudrun-require-project
[/external_agent_tool_result]

163. assistant (2026-07-02T06:11:33.125Z)

Branched from the latest `origin/master` (`239c8eb`, #344 just merged by a sibling). Let me read the current Gotchas section on this base and add the operator note:

164. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\README.md
[/external_agent_tool_call]

165. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_result]
52 ## Gotchas
53 - The dispatched container **must run inline** ? `CloudRunCompute` forces `compute_target=local_gpu`
54 + `indexing.detector_impl=native` in the per-execution args so the Job never re-dispatches itself.
55 - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud Run API status
56 ? so it reuses the storage layer + `execution_policy.poll_until` and survives a control-plane restart.
57 - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker** carrying the
58 real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min hang on a bad
59 video). Only a **hung/crashed** container (no marker at all) makes the bounded poll time out
60 (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and the API path
61 surfaces it as a `cloud dispatch error` job failure (not a traceback).
62 - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default libx264.
63
[/external_agent_tool_result]

166. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\README.md
[/external_agent_tool_call]

167. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

168. assistant (2026-07-02T06:11:33.125Z)

Doc note added. Now commit (explicit path), push, and open the PR:

169. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_call: Bash]
description: Commit doc change and verify branch isolation
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && echo "=== HEAD re-verify ===" && git branch --show-current && git log --oneline -1 && echo "=== stage explicit path ===" && git add deploy/cloudrun/README.md && git status --short deploy/ && cat > scratch/doc_commit.txt <<'EOF'
docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)

Document the operator-facing behaviour landed in #342: the real GoogleCloudRunJobClient.run_job now raises a clear RuntimeError when

```
GOOGLE_CLOUD_PROJECT is unset/empty (it must resolve a real job_path; an unset
project would build a bogus "projects/None/..." name). Add a Gotchas bullet so an
operator who hits the error knows where to set it.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
git commit -F scratch/doc_commit.txt 2>&1 | tail -3 && echo "=== only my commit vs
origin/master? ===" && git log --oneline origin/master..HEAD
[/external_agent_tool_call]
```

170. assistant (2026-07-02T06:11:33.125Z)

```
[external_agent_tool_result]
=== HEAD re-verify ===
docs/cloudrun-require-project
239c8eb fix(scripts): resolve gcloud via PATH in verify_compute_path so --verify-gcp works on
Windows (#344)
=== stage explicit path ===
M  deploy/cloudrun/README.md
[docs/cloudrun-require-project e5aebb0] docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required
(run_job fails fast if unset)
 1 file changed, 5 insertions(+)
=== only my commit vs origin/master? ===
e5aebb0 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)
[/external_agent_tool_result]
```

171. assistant (2026-07-02T06:11:33.125Z)

```
[external_agent_tool_call: Bash]
description: Push and create the doc PR
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && git push -u origin
docs/cloudrun-require-project 2>&1 | tail -4 && cat > scratch/doc_pr.md <<'EOF'
## What

Documents the operator-facing behaviour landed in [#342](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/342): `GoogleCloudRunJobClient.run_job` now fails fast with a clear
`RuntimeError` when `GOOGLE_CLOUD_PROJECT` is unset/empty (it must resolve a real `job_path`; an
unset project would otherwise build a bogus `projects/None/...` name).

Adds one bullet to the **Gotchas** section of `deploy/cloudrun/README.md` so an operator who
hits the error knows it's required and where to set it (step 3 already exports it).

Doc-only; no code change.

? Generated with [Claude Code](https://claude.com/claude-code)
EOF
gh pr create --base master --head docs/cloudrun-require-project --title "docs(cloudrun): note
GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)" --body-file scratch/doc_pr.md
2>&1 | tail -3
[/external_agent_tool_call]
```

172. assistant (2026-07-02T06:11:33.125Z)

```
[external_agent_tool_result]
remote:
branch 'docs/cloudrun-require-project' set up to track 'origin/docs/cloudrun-require-project'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      docs/cloudrun-require-project -> docs/cloudrun-require-project
https://github.com/Khelsutra/badminton-highlight-indexer/pull/345
[/external_agent_tool_result]
```

173. assistant (2026-07-02T06:11:33.125Z)

Doc PR [#345](https://github.com/Khelsutra/badminton-highlight-indexer/pull/345) created.

Watching CI to green, then I'll merge:

174. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_call: Bash]
description: Watch doc PR CI to completion
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && gh pr checks 345 --watch
--interval 30 2>&1 | tail -15; echo "=== watch exit: $? ==="
[/external_agent_tool_call]

175. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_result]
Command running in background with ID: b4zkvhk63. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\b4zkvhk63.output. You will be notified when
it completes. To check interim output, use Read on that file path.
[/external_agent_tool_result]

176. assistant (2026-07-02T06:11:33.125Z)

I'll wait for CI on the doc PR, then merge it.

177. user (2026-07-02T06:11:33.125Z)

<task-notification>
<task-id>b4zkvhk63</task-id>
<tool-use-id>toolu_01JwP12afBH4YYiLTTqdahKb</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\b4zkvhk63.output</output-file>
<status>completed</status>
<summary>Background command "Watch doc PR CI to completion" completed (exit code 0)</summary>
</task-notification>

178. assistant (2026-07-02T06:11:33.125Z)

CI passed (exit 0). Confirming all checks green and mergeable, then merging:

179. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_call: Bash]
description: Confirm doc PR checks and mergeability
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && gh pr checks 345 2>&1 |
head -15 && echo "=== mergeable ===" && gh pr view 345 --json mergeable,mergeStateStatus -q
'{mergeable: .mergeable, state: .mergeStateStatus}'
[/external_agent_tool_call]

180. assistant (2026-07-02T06:11:33.125Z)

[external_agent_tool_result]
Built wheel smoke (install + launch)   pass   1m11s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474368
Dependency license scan (no copyleft in runtime deps)   pass   32s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474435
Full test suite (offline, mocked)   pass   2m38s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474245
Golden fixtures cross-OS (macos-latest) pass   20s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474455
Golden fixtures cross-OS (ubuntu-latest)   pass   22s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474417
Golden fixtures cross-OS (windows-latest)   pass   51s
https://github.com/Khelsutra/badminton-highlight-

indexer/actions/runs/28016635693/job/82922474308
Leak scan (PII / attribution guard) pass 6s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28016635693/job/82922474579
Mypy (lenient baseline) pass 37s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28016635693/job/82922474256
Ruff lint pass 8s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28016635693/job/82922474310
Syntax sweep + import smoke (no GPU stack) pass 23s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28016635693/job/82922474268
=== mergeable ===
{"mergeable":"MERGEABLE","state":"CLEAN"}
[/external_agent_tool_result]

181. assistant (2026-07-02T06:11:33.126Z)

All 10 checks pass, state CLEAN. Merging and cleaning up:

182. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_call: Bash]
description: Squash-merge doc PR and refresh
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && gh pr merge 345 --squash --delete-branch 2>&1 | tail -4; echo "=== PR state ===" && gh pr view 345 --json state -q .state && git fetch origin --prune 2>&1 | tail -2 && echo "=== origin/master tip ===" && git log --oneline -1 origin/master
[/external_agent_tool_call]

183. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_result]
failed to run git: fatal: 'master' is already used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'

=== PR state ===
MERGED
 - [deleted] (none) -> origin/fix/verify-gcloud-windows
   239c8eb..9329473 master -> origin/master
=== origin/master tip ===
9329473 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset) (#345)
[/external_agent_tool_result]

184. assistant (2026-07-02T06:11:33.126Z)

PR **#345 MERGED** to `master` as `9329473`. The git error was again only gh's local-checkout switch (master held by the worktree) ? harmless. Let me clean up git state, then add the #345 citation to the memory notes:

185. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_call: Bash]
description: Detach to origin/master, delete merged branch, clean scratch
command: cd "C:/Users/avidu/Projects/badminton-highlight-indexer" && git checkout --detach origin/master 2>&1 | tail -1 && git branch -D docs/cloudrun-require-project 2>&1 | tail -1 && rm -f scratch/doc_commit.txt scratch/doc_pr.md && echo "=== final HEAD ===" && git log --oneline -1 && echo "=== clean tracked tree? ===" && git status --short backend/ tests/ deploy/
[/external_agent_tool_call]

186. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_result]
HEAD is now at 9329473 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset) (#345)

Deleted branch docs/cloudrun-require-project (was e5aebb0).
=== final HEAD ===
9329473 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)
(#345)
=== clean tracked tree? ===
[/external_agent_tool_result]

187. assistant (2026-07-02T06:11:33.126Z)

Now cite #345 in the memory notes so the record is precise:

188. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md
[/external_agent_tool_call]

189. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

190. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]

191. assistant (2026-07-02T06:11:33.126Z)

[external_agent_tool_result]
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

192. assistant (2026-07-02T06:11:33.126Z)

All updates are landed. Here's the final state.

Memories, handoff & docs ? updated to reflect the completed work

Two PRs merged (both squash-merged, branched from `origin/master`, verified-green, per the merge working agreement):
- [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342) (`688d28f`) ? the `GOOGLE_CLOUD_PROJECT` guard + test (merged earlier this turn)
- [#345](https://github.com/Khelsutra/badminton-highlight-indexer/pull/345) (`9329473`) ? the operator doc note (just merged)

Memory (`~/.claude/.../memory/`):
- [current-state.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md) ? replaced the sibling's brief "#342 chip" stub with the full record: the latent bug, the fix mechanism, the test, suite/coverage numbers, and the stale-`.mypy_cache` gotcha; added the #345 follow-up.
- [session-handoff.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md) ? added a `(2026-06-23 #342)` "You are here" entry citing both PRs.

Docs (in-repo, shipped via #345):
- `deploy/cloudrun/README.md` ? a "Gotchas" bullet documenting that `GOOGLE_CLOUD_PROJECT` is required and `run_job` now fails fast if unset, so an operator who hits the error knows where to set it.

Notes on scope

- **No new dedicated memory file**: a merged bug-fix is git history, which the memory-hygiene rules say not to duplicate ? the checkpoint pointers in current-state/handoff are the right record.
- **`MEMORY.md` index unchanged**: no new memory files, so no new index line needed.
- **Concurrency-safe**: master moved twice under me during this (siblings merged #344, then my #345); I re-cut from `origin/master` each time, verified `origin/master..HEAD` contained only my commit, and staged explicit paths. Final `origin/master` = `9329473`. Local tree clean; merged branches deleted; scratch files removed.

Both fixes are live on `master` and the session record is current.